

Instruction Composition Refactor

Goal

Keep instruction authoring simple in Markdown while eliminating duplicated content in Redis and removing the need to assemble complete instructions on the client.

Design

Each task instruction remains a normal Markdown file containing:

- YAML frontmatter describing its dependencies.
- A Markdown body containing only the task-specific instruction.

Example:

```
---
slug: ins.hhp-normalization
role: "[[Editor Role]]"
context:
  - "[[General Tone and Style]]"
  - "[[HHP Project Context]]"
---

Normalize spelling, punctuation, capitalization and house style while preserving
meaning.
```

The task instruction should never contain copies of the role or project context.

Upload workflow

Instruction uploader

1. Read the instruction file.
2. Parse the YAML frontmatter directly.
3. Extract:
 4. instruction slug
 5. role wikilink
 6. context wikilinks
7. Resolve the wikilinks to instruction slugs.
8. Remove the YAML frontmatter.

9. Write only the Markdown body to a temporary file.
10. Run Pandoc **once** on the temporary file.
11. Generate the instruction hash from the Pandoc output.
12. Upload only the task instruction record.

The uploaded instruction therefore contains only the reusable task instruction.

Plan upload

When uploading a plan, the uploader copies the resolved dependency slugs into each step.

Example:

```
{
  "ordinal": 2,
  "engine": "llm",
  "instruction_slug": "ins.hhp-normalization",
  "role_slug": "ins.editor-role",
  "context_slugs": [
    "ins.general-tone-style",
    "ins.hhp-project-context"
  ]
}
```

No instruction text is duplicated.

Plan ingestion

When the server materializes the plan, it resolves these slugs into instruction identities (or keys).

Each step therefore contains references such as

```
instruction_key
role_key
context_keys[]
```

The step stores only references.

Execution

Immediately before calling the LLM:

1. Fetch the role instruction.
2. Fetch all context instructions.

3. Fetch the task instruction.
4. Assemble the message stack.
5. Execute the model.

Composition happens only at runtime.

Benefits

No duplicated Redis content

Role and context exist exactly once.

Every task instruction simply references them.

Single Pandoc invocation

The uploader strips the frontmatter before conversion.

No second Pandoc pass is required merely to generate a hash.

Stable instruction identities

The instruction hash represents only the task instruction.

Updating the referenced role or project context does not create a new task instruction.

Better separation of concerns

Markdown authoring defines composition.

The uploader resolves links.

The plan records dependencies.

The engine performs final assembly.

Each layer has one clear responsibility.

Implementation notes

- Parse frontmatter directly instead of using Pandoc.
- Split the document by locating the closing YAML fence.
- Fail loudly if either fence is missing.
- Generate the temporary Markdown body after removing frontmatter.
- Run Pandoc exactly once.
- Keep existing Markdown authoring workflow unchanged.

Result

Instruction files become small, reusable components.

Plans carry dependency references rather than duplicated text.

Redis stores each reusable instruction exactly once.

The execution engine assembles the complete prompt only when the model is invoked.